

**LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 6 ACARA 14
SEMESTER 3**



**Tugas :
Security, Exception, Logging, PRG, dan
Integrasi CRUD Lengkap**

**Disusun Oleh:
Uaryl Engno Laurenca (E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. PENDAHULUAN

Pada praktikum sebelumnya (Acara 13), aplikasi Sistem Informasi Akademik sudah menerapkan Service Layer untuk memisahkan logika validasi dari Controller. Namun aplikasi masih memiliki beberapa kelemahan yang berpotensi membahayakan keamanan dan kenyamanan pengguna, di antaranya input yang belum divalidasi secara menyeluruh, query yang rawan SQL Injection, pesan error teknis dari database yang langsung ditampilkan ke pengguna, tidak adanya pencatatan (log) ketika terjadi error, serta kemungkinan form terkirim ulang saat halaman di-refresh.

Pada Acara 14 ini, aplikasi diperbaiki dengan menerapkan lima aspek utama: Security (validasi input dan prepared statement), Exception Handling (try-catch pada proses yang berpotensi error), Logging (pencatatan error ke file `storage/logs/app.log`), PRG/Post-Redirect-Get (mencegah pengiriman form berulang), dan Integrasi CRUD lengkap dengan MySQL yang mengikat seluruh konsep tersebut menjadi satu alur yang aman.

B. TUJUAN PRAKTIKUM

1. Mahasiswa mampu menerapkan validasi input dan prepared statement untuk mencegah SQL Injection.
2. Mahasiswa mampu menerapkan exception handling (try-catch) pada proses CRUD.
3. Mahasiswa mampu membuat sistem logging untuk mencatat error ke file log.
4. Mahasiswa mampu menerapkan pola PRG agar form tidak terkirim ulang saat refresh.
5. Mahasiswa mampu mengintegrasikan seluruh konsep tersebut ke dalam CRUD Data Mahasiswa yang terhubung dengan MySQL.

C. DASAR TEORI

Security pada Aplikasi Web diterapkan melalui validasi input pengguna sebelum data diproses serta penggunaan *prepared statement* pada setiap query SQL. Dengan pendekatan ini, perintah SQL dipisahkan dari data yang diberikan pengguna sehingga dapat mencegah terjadinya SQL Injection. Selain itu, Exception Handling menggunakan blok `try-catch` untuk menangani kondisi program yang bermasalah secara terkendali, seperti kegagalan koneksi database, tanpa menampilkan pesan error teknis secara langsung kepada pengguna. Logging juga digunakan untuk mencatat kejadian penting atau error ke dalam file sehingga dapat membantu proses penelusuran dan perbaikan oleh programmer, dengan tetap menghindari pencatatan informasi sensitif seperti password database.

PRG (Post/Redirect/Get) diterapkan untuk mencegah pengiriman ulang form ketika pengguna melakukan refresh setelah proses submit berhasil. Setelah request POST selesai diproses, aplikasi melakukan *redirect* menuju halaman GET sehingga refresh hanya menjalankan kembali request GET. Seluruh konsep tersebut kemudian diintegrasikan dalam CRUD Lengkap dengan alur *User → Controller → Service → Validasi → Repository → MySQL → Exception jika terjadi error → Logging → Flash Message → Redirect (PRG) → User melihat hasil*. Integrasi ini membuat

aplikasi lebih aman, terstruktur, mudah ditelusuri ketika terjadi kesalahan, serta memberikan pengalaman penggunaan yang lebih baik.

D. HASIL DAN PEMBAHASAN

1. Struktur Database

```
1 CREATE TABLE mahasiswa (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     nim VARCHAR(20) NOT NULL UNIQUE,  
4     nama VARCHAR(100) NOT NULL,  
5     email VARCHAR(100) NOT NULL,  
6     prodi_id INT NOT NULL,  
7     angkatan YEAR NOT NULL  
8 );
```

Tabel mahasiswa dibuat/diperbarui dengan struktur berikut agar mendukung relasi ke prodi dan tahun angkatan:

2. Koneksi Database yang Aman

```
ACARA14 > app > Core > Database.php  
1 <?php  
2  
3 namespace App\Core;  
4  
5 use PDO;  
6 use PDOException;  
7  
8 class Database  
9 {  
10     private static ?PDO $instance = null;  
11  
12     public static function getInstance(): PDO  
13     {  
14         if (self::$instance === null) {  
15             try {  
16                 $host = 'localhost';  
17                 $db = 'si_akademik_webserver';  
18                 $user = 'root';  
19                 $pass = '';  
20  
21                 self::$instance = new PDO(  
22                     "mysql:host={$host};dbname={$db};charset=utf8mb4",  
23                     $user,  
24                     $pass,  
25                     [  
26                         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
27                         PDO::ATTR_EMULATE_PREPARES => false,  
28                     ]  
29                 );  
30             } catch (PDOException $e) {  
31                 Logger::error('Koneksi database gagal: ' . $e->getMessage());  
32                 die('Terjadi kesalahan pada server. Silakan coba lagi nanti.');33             }  
34         }  
35  
36         return self::$instance;  
37     }  
38 }
```

Database.php diubah agar tidak lagi menampilkan pesan error PDO secara langsung ke pengguna. Jika koneksi gagal, detail error dicatat ke log melalui `Logger::error()`, dan pengguna hanya melihat pesan generik "Terjadi kesalahan pada server."

3. Sistem Logging

```
ACARA14 > app > Core > Logger.php
1  <?php
2
3  namespace App\Core;
4
5  class Logger
6  {
7      private static function path(): string
8      {
9          return __DIR__ . '/../storage/logs/app.log';
10     }
11
12     public static function error(string $message): void
13     {
14         $line = date('Y-m-d H:i:s') . ' - ' . $message . PHP_EOL;
15         error_log($line, 3, self::path());
16     }
17
18     public static function info(string $message): void
19     {
20         $line = date('Y-m-d H:i:s') . ' - INFO - ' . $message . PHP_EOL;
21         error_log($line, 3, self::path());
22     }
23 }
```

Dibuat class `Logger` baru pada `app/Core/Logger.php` yang mencatat pesan error beserta timestamp ke file `storage/logs/app.log` menggunakan `error_log()` dengan mode 3 (menulis ke file).

4. Exception Handling pada Service Layer

```
1 <?php
2
3 namespace App\Services;
4
5 use App\Repositories\MahasiswaRepository;
6 use App\Repositories\ProdiRepository;
7 use App\Core\Logger;
8 use Exception;
9
10 class MahasiswaService
11 {
12     private MahasiswaRepository $mahasiswaRepository;
13     private ProdiRepository $prodiRepository;
14
15     public function __construct(
16         MahasiswaRepository $mahasiswaRepository,
17         ProdiRepository $prodiRepository
18     ) {
19         $this->mahasiswaRepository = $mahasiswaRepository;
20         $this->prodiRepository = $prodiRepository;
21     }
22
23     public function list(): array
24     {
25         try {
26             return $this->mahasiswaRepository->all();
27         } catch (\Throwable $e) {
28             Logger::error('Gagal mengambil data mahasiswa: ' . $e->getMessage());
29             return [];
30         }
31     }
32
33     public function find(int $id)
34     {
35         try {
36             return $this->mahasiswaRepository->find($id);
37         } catch (\Throwable $e) {
38             Logger::error('Gagal mengambil mahasiswa id-' . $id . ': ' . $e->getMessage());
39             return null;
40         }
41     }
42
43     public function prodiOptions(): array
44     {
45         try {
46             return $this->prodiRepository->all();
47         } catch (\Throwable $e) {
48             Logger::error('Gagal mengambil data prodi: ' . $e->getMessage());
49             return [];
50         }
51     }
52
53     public function create(array $input): array
54     {
55         $errors = $this->validate($input);
56
57         if (empty($errors)) {
58             return ['success' => false, 'message' => $errors[0]];
59         }
60
61         try {
62             $this->mahasiswaRepository->create($input);
63             return ['success' => true, 'message' => 'Data mahasiswa berhasil ditambahkan.'];
64         } catch (\Throwable $e) {
65             Logger::error('Gagal menambah mahasiswa: ' . $e->getMessage());
66             return ['success' => false, 'message' => 'Data gagal disimpan.'];
67         }
68     }
69
70     public function update(int $id, array $input): array
71     {
72         $errors = $this->validate($input, $id);
73
74         if (empty($errors)) {
75             return ['success' => false, 'message' => $errors[0]];
76         }
77
78         try {
79             $this->mahasiswaRepository->update($id, $input);
80             return ['success' => true, 'message' => 'Data mahasiswa berhasil diubah.'];
81         } catch (\Throwable $e) {
82             Logger::error('Gagal mengubah mahasiswa id-' . $id . ': ' . $e->getMessage());
83             return ['success' => false, 'message' => 'Data gagal disimpan.'];
84         }
85     }
86
87     public function delete(int $id): array
88     {
89         try {
90             $this->mahasiswaRepository->delete($id);
91             return ['success' => true, 'message' => 'Data mahasiswa berhasil dihapus.'];
92         } catch (\Throwable $e) {
93             Logger::error('Gagal menghapus mahasiswa id-' . $id . ': ' . $e->getMessage());
94             return ['success' => false, 'message' => 'Data gagal dihapus.'];
95         }
96     }
97
98     private function validate(array $input, ?int $ignoreId = null): array
99     {
100         $errors = [];
101
102         $nim = trim($input['nim'] ?? '');
103         $nama = trim($input['nama'] ?? '');
104         $email = trim($input['email'] ?? '');
105         $prodiId = (int)($input['prodi_id'] ?? 0);
106
107         if ($nim === '') {
108             $errors[] = 'NIM wajib diisi.';
109         } elseif (!ctype_digit($nim)) {
110             $errors[] = 'NIM harus berupa angka.';
111         } elseif ($this->mahasiswaRepository->existsByNim($nim, $ignoreId)) {
112             $errors[] = 'NIM sudah terdaftar.';
113         }
114
115         if ($nama === '') {
116             $errors[] = 'Nama wajib diisi.';
117         }
118
119         if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
120             $errors[] = 'Format email tidak valid.';
121         }
122
123         if ($prodiId <= 0 || !$this->prodiRepository->exists($prodiId)) {
124             $errors[] = 'Program studi tidak valid.';
125         }
126
127         return $errors;
128     }
129 }
```

`MahasiswaService` dibungkus dengan blok `try-catch` pada setiap operasi yang menyentuh `Repository` (`create`, `update`, `delete`, `list`, `find`). Jika terjadi `exception`, `Service` menangkapnya, mencatat detail error ke log, dan mengembalikan pesan yang aman seperti "Data gagal disimpan." tanpa membocorkan informasi teknis database.

5. Penerapan PRG pada Controller

SI Akademik						Mahasiswa	Prodi	Mata Kuliah
Data Mahasiswa						+ Tambah Mahasiswa		
Cari nama atau NIM...						Cari		
NIM	Nama	Email	Prodi	Angkatan	Aksi			
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus			

Setiap aksi `store()`, `update()`, dan `destroy()` pada `MahasiswaController` diakhiri dengan `redirect('/mahasiswa')` setelah proses POST selesai, bukan menampilkan view secara langsung. Dengan demikian, ketika pengguna menekan F5 setelah submit, browser hanya mengulang request GET terakhir (halaman daftar mahasiswa), bukan mengirim ulang data form.

6. Flash Message

SI Akademik						Mahasiswa	Prodi	Mata Kuliah
Data mahasiswa berhasil ditambahkan.								
Data Mahasiswa						+ Tambah Mahasiswa		
Cari nama atau NIM...						Cari		
NIM	Nama	Email	Prodi	Angkatan	Aksi			
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus			

SI Akademik						Mahasiswa	Prodi	Mata Kuliah
NIM sudah terdaftar.								
Data Mahasiswa						+ Tambah Mahasiswa		
Cari nama atau NIM...						Cari		
NIM	Nama	Email	Prodi	Angkatan	Aksi			
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus			

Setiap hasil proses (berhasil/gagal) dikirim melalui `$_SESSION['flash']` dan ditampilkan satu kali di halaman tujuan, kemudian dihapus (`unset`) agar tidak muncul lagi saat halaman direfresh ulang.

7. Pengujian Skenario

No	Skenario	Hasil
1	Tambah mahasiswa dengan data valid	Redirect ke /mahasiswa, flash hijau "Data mahasiswa berhasil ditambahkan."
2	Tambah mahasiswa dengan NIM kosong	Flash merah "NIM wajib diisi."
3	Tambah mahasiswa dengan NIM yang sudah terdaftar	Flash merah "NIM sudah terdaftar."
4	Masukkan email yang tidak valid	Flash merah "Format email tidak valid."
5	Edit data mahasiswa	Data tersimpan, flash hijau "Data mahasiswa berhasil diubah."
6	Hapus data mahasiswa	Flash hijau "Data mahasiswa berhasil dihapus."
7	Refresh halaman setelah proses tambah	Tidak ada pengiriman ulang form, data tidak doble
8	Simulasikan error database (MySQL dimatikan)	Flash merah generik "Data gagal disimpan.", tanpa detail teknis
9	Periksa storage/logs/app.log setelah error database	Tercatat baris log dengan timestamp dan pesan error
10	Pastikan tidak ada informasi teknis database yang tampil ke pengguna	Tidak ditemukan teks SQLSTATE atau nama tabel/kolom di halaman manapun

```

ACARA14 > storage > logs > app.log
1 2026-10-04 08:45:24 - Koneksi database gagal: SQLSTATE[HY000] [1049] Unknown database 'si_akademik'
2 2026-10-04 08:45:35 - Koneksi database gagal: SQLSTATE[HY000] [1049] Unknown database 'si_akademik'
3 2026-10-04 08:47:46 - Koneksi database gagal: SQLSTATE[HY000] [1049] Unknown database 'si_akademik'
4 2026-10-04 11:32:14 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
5 2026-10-04 11:33:33 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
6 2026-10-04 11:33:43 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
7 2026-10-04 11:34:44 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
8 2026-10-04 11:35:09 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
9 2026-10-04 11:36:55 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
10 2026-10-04 11:36:56 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
11 2026-10-04 11:37:26 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
12 2026-10-04 11:37:43 - Gagal mengambil data mahasiswa: Too few arguments to function App\Models\Mahasiswa::__construct(), 0 passed in C:\xampp\htdocs\BKPM-Web-1
13 |

```

E. KESIMPULAN

Melalui Acara 14 ini, mahasiswa memahami bahwa aplikasi yang baik tidak hanya mampu menyimpan data, tetapi juga harus aman, tervalidasi, mampu menangani error dengan baik, mencatat error untuk keperluan debugging, memberikan feedback yang jelas kepada pengguna, serta mencegah pengiriman form berulang. Dengan menggabungkan validasi input, prepared statement, exception handling, logging, flash message, dan PRG, aplikasi CRUD Data Mahasiswa menjadi lebih andal dan aman digunakan, sekaligus mencerminkan alur kerja aplikasi web yang profesional: *User → Controller → Service → Validasi → Repository → MySQL → Exception → Logging → Flash Message → Redirect (PRG) → User melihat hasil.*

